

YOLO26n-Based Bowling-scoring System for RC-Car

Abdelrahman Abdelrassoul - 202300056

May 7, 2026

Abstract

This report details the development, training, and deployment of an offline, mobile-based computer vision system designed to analyze RC-car bowling. The system utilizes a lightweight YOLO26n object detection model, heavily optimized for edge deployment, coupled with a custom React Native and Kotlin-based mobile application architecture. By strictly separating the UI layer from a high-performance native computer vision backend, the application performs fully offline video decoding, inference, tracking, and video encoding directly on an Android device. The model achieved a mean Average Precision (mAP50) of 0.915, successfully detecting standing pins, fallen pins, bounding balls, and RC cars, while a custom rule-based temporal tracking algorithm robustly scores pin-fall sequences and calculates car trajectories.

Contents

1	Introduction	3
2	Dataset and Model Training	3
2.1	Dataset Preparation	3
2.2	Data Augmentation	3
2.3	Model Architecture and Transfer Learning	3
2.4	Training Configuration	3
3	Mobile Application Architecture	4
3.1	Application Layers	4
4	Computer Vision Pipeline	4
4.1	Video Recording and Decoding	4
4.2	Preprocessing and Inference	4
4.3	Output Video Generation	5
5	Tracking and Scoring Logic	5
5.1	Pin State Analysis	5
5.2	Car Trajectory Rendering	5
6	Deployment	5
7	Results	5
7.1	Model Accuracy	5
7.2	Inference Speed	6
8	Challenges	6
9	Limitations	6
10	Future Work	6
11	Conclusion	7

1 Introduction

The objective of this project was to develop a lightweight, edge-deployable object detection and analysis pipeline capable of evaluating RC-car bowling runs in real time. The resulting system captures video natively on a mobile device, detects bowling-related objects, tracks pin states over time, detects standing-to-fallen transitions, and renders annotated outputs entirely offline.

Achieving this required solving two major challenges: developing a highly efficient object detector capable of parsing small objects under dynamic indoor motion, and engineering a mobile application architecture capable of processing frame-by-frame computer vision tasks without blocking the user interface thread or relying on cloud inference.

2 Dataset and Model Training

2.1 Dataset Preparation

The dataset was collected and annotated using Roboflow, sourcing imagery from mobile-recorded videos of indoor toy-car bowling environments. The dataset defines four primary object classes: *ball*, *car*, *fallen-pins*, and *standing-pins*.

The collected dataset was reorganized into a YOLO-compatible structure and split into Training (70%), Validation (20%), and Testing (10%) subsets. The training set comprised 29,159 bounding boxes (predominantly standing-pins), the validation set contained 3,045 boxes, and the test set contained 1,336 boxes.

2.2 Data Augmentation

To improve model generalization against variations in lighting and camera angles, automatic augmentations were applied during training via the Ultralytics pipeline. These included mosaic augmentation (1.0), translation (0.1), scaling (0.5), horizontal flipping (0.5), alongside HSV color adjustments, blur, and random erasing.

2.3 Model Architecture and Transfer Learning

The YOLO26n architecture was selected due to its nano-scale footprint, making it ideal for fast edge inference and low memory consumption. The final fused inference model contains 122 layers, 2.37 million parameters, and requires only 5.2 GFLOPs.

To accelerate convergence, transfer learning was utilized. The initialization transferred 606 out of 708 pretrained YOLO26n layers, allowing the model to leverage low-level visual features, reduce overall training time, and significantly improve the detection of small objects within the frame.

2.4 Training Configuration

Training was executed on a Tesla T4 GPU using CUDA 12.8 and Ultralytics 8.4.46. The model was trained over a maximum of 100 epochs using the AdamW optimizer (initial learning rate of 0.001 with Cosine LR scheduling and a weight decay of 0.0005). Mixed

precision training was enabled to optimize GPU memory utilization. Early stopping was triggered after 25 epochs of no improvement, with the best weights recorded at epoch 71.

3 Mobile Application Architecture

The mobile client employs a hybrid architecture designed to maintain a fluid React Native UI while delegating computationally expensive tasks to native compiled code.

3.1 Application Layers

The application is strictly separated into distinct functional tiers:

- **UI Layer:** Developed in React Native and TypeScript, managing user interactions, camera previews (via `expo-camera`), and results playback.
- **Orchestration Layer:** A TypeScript-based state management tier coordinating transitions and data parsing.
- **React Native Bridge:** Exposes native Kotlin functions to the JavaScript runtime via the Expo Modules API, ensuring non-blocking asynchronous communication.
- **Native Android CV Layer:** A Kotlin-based backend utilizing Android Media APIs for low-level video decoding, frame sampling, and encoding.
- **Model Inference Layer:** Dedicated to loading and executing predictions using the TensorFlow Lite (TFLite) Android runtime.
- **Tracking and Scoring Layer:** Evaluates raw model spatial outputs over sequential frames to track objects and identify state transitions.
- **Storage Layer:** Manages local read/write operations for raw videos, annotated outputs, and JSON metadata.

4 Computer Vision Pipeline

4.1 Video Recording and Decoding

The pipeline initiates offline when the user captures a live video. The raw `.mp4` file is stored locally, and its URI is passed to the native Kotlin module. Inside the native module, a `VideoDecoder` extracts frames at a controlled sampling rate (e.g., 10–15 FPS) to maintain a stable processing load without overwhelming mobile hardware.

4.2 Preprocessing and Inference

A `FramePreprocessor` resizes the extracted bitmaps and applies color space conversions to match the YOLO model’s expected input tensor shape. The `TFLiteDetector` then executes the model using either the CPU or a hardware-accelerated GPU delegate. The output consists of bounding box coordinates, class identifications, and confidence scores for every processed frame.

4.3 Output Video Generation

Following tracking and scoring, an `AnnotatedVideoWriter` uses Android Canvas or OpenCV drawing functions to superimpose dynamic data onto the original frames, including hit-order numeric labels and the car trajectory. The annotated frames are sequentially pushed through an Android `MediaCodec` and `MediaMuxer` pipeline to generate a final, compressed .mp4 video file.

5 Tracking and Scoring Logic

5.1 Pin State Analysis

Because bowling pins are initially stationary, the system employs a deterministic, rule-based tracking algorithm instead of computationally heavy deep-learning trackers. A `PinTracker` matches frame-by-frame detections using Intersection over Union (IoU) and bounding box center-distance thresholds to maintain stable track IDs.

Scoring relies on temporal hysteresis logic: a pin is registered as "fallen" only when its classification shifts from `standing_pin` to `fallen_pin` and remains stable above a designated confidence threshold for multiple consecutive frames. The hit order is assigned sequentially based on the initial timestamp of each stable fall event.

5.2 Car Trajectory Rendering

Path tracking isolates the center point of the `car` bounding box across sequential frames. To mitigate noise and missing detections caused by motion blur, a moving average mathematical smoothing algorithm is applied to the coordinate history before rendering the trajectory as a continuous visual line.

6 Deployment

Deploying deep learning models to edge devices necessitates careful formatting. Multiple deployment formats were evaluated, including quantized TFLite and Float16 TFLite models. Extreme int8 quantization produced unacceptable accuracy degradation and unstable predictions regarding pin states. Consequently, the final deployment utilized standard Float32 TensorFlow Lite conversion. This format successfully balanced memory footprint reduction with the preservation of pin-state accuracy and confidence reliability. The application was compiled using a Custom EAS Development Build to package the native ML binaries alongside the React Native JS bundle.

7 Results

7.1 Model Accuracy

The best model achieved strong detection metrics across the test set, yielding an overall Precision of 0.909, a Recall of 0.880, an mAP50 of 0.915, and an mAP50-95 of 0.744.

Per-class performance varied based on visual complexity:

- **standing-pins:** mAP50 of 0.963
- **car:** mAP50 of 0.915
- **fallen-pins:** mAP50 of 0.894
- **ball:** mAP50 of 0.886

Standing pins achieved the highest accuracy due to higher dataset frequency and clear spatial structures, whereas fallen pins presented a harder challenge due to varied orientations and occlusion.

7.2 Inference Speed

The YOLO26n architecture demonstrated exceptional validation speeds on edge hardware, requiring approximately 0.2 ms for preprocessing, 2.2 ms for inference, and 2.3 ms for postprocessing, proving highly suitable for mobile deployment.

8 Challenges

Several technical hurdles were addressed during development:

- **Duplicate Detections:** The model occasionally produced overlapping bounding boxes on the same pin. This was mitigated by tuning Non-Maximum Suppression (NMS) thresholds and introducing duplicate suppression handling in the tracking layer.
- **False State Transitions:** Standing pins were occasionally classified as fallen due to partial occlusion or motion blur. The solution was the implementation of the multi-frame temporal hysteresis requirement before confirming a fall event.
- **React Native Limitations:** JavaScript bridge bottlenecks prevented real-time frame manipulation. This was solved by delegating the entire video-to-video pipeline to natively compiled Kotlin APIs.

9 Limitations

Despite successful deployment, the system maintains certain limitations. Fallen pins remain inherently more difficult to detect than standing pins due to severe visual occlusion when clustered. Furthermore, highly rapid motion of the RC car or the bowling ball can cause temporary pin-state tracking inconsistencies and discontinuous trajectory plotting.

10 Future Work

Future enhancements to the system could include:

- Expansion of the dataset to encompass a wider variety of lighting conditions and camera angles.
- Exploration of transformer-based object tracking or dedicated re-identification modules to handle severe occlusion natively.

- Application of quantization-aware training (QAT) to allow for effective int8 TFLite compression without the observed accuracy degradation.
- Deeper optimization of mobile GPU delegates and Neural Processing Unit (NPU) utilization on modern Android devices.

11 Conclusion

This project successfully developed a lightweight YOLO26n-based object detection system integrated into a high-performance Android mobile application for RC-car bowling analysis. By strictly separating the UI layer from a Kotlin-based native media processing backend, the system achieves fully offline execution. The final model yielded an mAP50 of 0.915 and demonstrated real-time inference speeds. The pipeline effectively tracks objects, classifies temporal pin states, determines fall order, and generates dynamically annotated output videos entirely on the edge device, proving the viability of deploying complex deep-learning tracking logic within mobile-constrained environments.